E-Land Records Management System: A Blockchain-Based Land Administration Platform

Submitted by:

Byreddy Rajesh - 23MCF1R08
Kulakarni Shiva - 23MCF1R21
Kampasati Koteswara Rao - 23MCF1R18

Under the guidance of

Pro. E. Suresh Babu
Department of Computer Science and Engineering
National Institute of Technology, Warangal

November 24, 2025

Abstract

This comprehensive report presents the design, implementation, and deployment of an E-Land Records Management System built on Hyperledger Fabric blockchain technology. The system provides a secure, transparent, and efficient platform for land record management with role-based access control across multiple government organizations. The implementation includes a Next.js web application, Node.js API server, and custom chaincode for blockchain operations, ensuring immutable audit trails and decentralized data integrity.

Contents

1 Introduction

1.1 Project Overview

The E-Land Records Management System is a comprehensive digital solution for land administration that leverages blockchain technology to ensure transparency, security, and immutability of land records. The system addresses critical challenges in traditional land management systems including data tampering, lack of transparency, and inefficient approval workflows.

1.2 Objectives

- Implement a secure land records management system using Hyperledger Fabric
- Provide role-based access control for different government officials
- Ensure data integrity and immutability through blockchain technology
- Create an intuitive web interface for users and officials
- Enable efficient document management with IPFS integration
- Implement automated workflow for land application processing

1.3 Key Features

- **Dual Authentication System**: Separate login portals for citizens and government officials
- **Role-Based Access Control**: 11 distinct official roles with specific permissions
- **Blockchain Integration**: All critical operations recorded on Hyperledger Fabric
- **Document Management**: IPFS integration for secure document storage
- **Real-time Workflow**: Automated approval hierarchy with status tracking
- **Patta Certificate Generation**: Automated certificate creation and distribution
- **Mobile-Responsive Design**: Accessible across all devices

2 System Architecture

2.1 Overall Architecture

The system follows a three-tier architecture with clear separation of concerns:

1. **Presentation Layer**: Next.js web application with React components
2. **Application Layer**: Express.js API servers for business logic
3. **Data Layer**: MongoDB for operational data and Hyperledger Fabric for immutable records

2.2 Technology Stack

2.2.1 Frontend Technologies

- **Next.js 16.0.3**: React framework for server-side rendering and API routes
- **React 19.2.0**: Component-based UI development
- **Tailwind CSS 4.0**: Utility-first CSS framework
- **TypeScript 5.0**: Type-safe JavaScript development
- **React Icons**: Icon library for UI components

2.2.2 Backend Technologies

- **Node.js**: Runtime environment for server-side applications
- **Express.js**: Web application framework
- **MongoDB/Mongoose**: NoSQL database and ODM
- **Hyperledger Fabric 2.5**: Enterprise blockchain platform
- **Fabric SDK**: Client libraries for blockchain interaction

2.2.3 Supporting Technologies

- **IPFS/Pinata**: Decentralized file storage
- **Puppeteer**: PDF generation for certificates
- **Nodemailer**: Email notifications
- **Bcrypt**: Password hashing
- **JWT**: Session management

2.3 Network Architecture

The Hyperledger Fabric network consists of three organizations representing different government hierarchies:

- **Org1**: Administrative roles (Clerk, Superintendent, Project Officer)
- **Org2**: Field operations (MRO, Surveyor, Revenue Inspector, VRO)
- **Org3**: Executive roles (Revenue Dept Officer, Joint Collector, District Collector, Ministry Welfare)

3 Database Design

3.1 MongoDB Collections

3.1.1 User Collection

```
{
  _id: ObjectId,
  firstName: String,
  middleName: String,
  lastName: String,
  email: String (unique),
  phoneNumber: String,
  password: String (hashed),
  dateOfBirth: Date,
  createdAt: Date,
  updatedAt: Date
}
```

3.1.2 Official Collection

```
{
  _id: ObjectId,
  firstName: String,
  lastName: String,
  email: String (unique),
  phoneNumber: String,
  password: String (hashed),
  designation: String,
  officeId: String,
  createdAt: Date,
  updatedAt: Date
}
```

3.1.3 LandRequest Collection

```
{
  _id: ObjectId,
  txnId: String,
  nature: String,
  createdBy: String,
  // Personal Details
  fullName: String,
  email: String,
  phoneNumber: String,
  aadharNumber: String,
  dob: Date,
  // Land Details
  ownerName: String,
  surveyNumber: String,
  area: String,
  address: String,
  state: String,
  city: String,
  pincode: String,
```

```
  // Documents
  ipfsHash: String,
  receiptNumber: String,
  // Status
  status: String,
  currentlyWith: String,
  // Patta Details
  pattaHash: String,
  pattaNumber: String,
  certificateNumber: String,
  pattaGeneratedAt: Date,
  // Survey Data
  surveyData: Object,
  fieldPhotos: Array,
  surveyRemarks: String,
  // Action History
  actionHistory: Array,
  createdAt: Date,
  updatedAt: Date
}
```

3.2 Blockchain Data Structure

The blockchain stores land request data with the following structure:

```
{
  "receiptNumber": "LR-ABC123",
  "status": "completed",
  "currentlyWith": "official_id",
  "createdAt": "2025-11-24T10:00:00.000Z",
  "history": [
    {
      "txn_id": "TXN-123456",
      "officialId": "official_id",
      "officialName": "Official Name",
      "designation": "clerk",
      "action": "approved",
      "remarks": "Approved by clerk",
      "timestamp": "2025-11-24T10:00:00.000Z",
      "data": {}
    }
  ]
}
```

4 Chaincode Implementation

4.1 Chaincode Structure

The land records chaincode is implemented in Node.js using Fabric Contract API:

```
const { Contract } = require('fabric-contract-api');

class LandRecordContract extends Contract {
    // Contract methods
}
```

4.2 Core Functions

4.2.1 createLandRequest

Creates a new land request on the blockchain:

- Validates receipt number uniqueness
- Parses and validates input data
- Initializes status and history
- Stores data immutably on blockchain

4.2.2 updateLandStatus

Updates the status of a land request:

- Verifies request existence
- Updates status and assignee
- Adds action to history with transaction ID
- Maintains audit trail

4.2.3 readLandRequest

Retrieves land request data:

- Fetches data by receipt number
- Returns complete request information
- Includes full action history

4.2.4 Query Functions

- **getAllLandRequests**: Returns all land requests
- **getLandRequestsByStatus**: Filters by status
- **getLandRequestsByAssignee**: Filters by current assignee
- **getLandRequestHistory**: Returns action history

4.2.5 Document Management

- **storeDocumentHash**: Stores IPFS hash on blockchain
- **verifyDocument**: Verifies document integrity
- **issuePattaCertificate**: Records certificate issuance
- **verifyPattaCertificate**: Verifies certificate authenticity

4.3 CouchDB Integration

The chaincode includes CouchDB indexes for efficient querying:

```
{
  "index": {
    "fields": ["status"]
  },
  "name": "status-index",
  "type": "json"
}
```

5 API Layer Implementation

5.1 Fabric-API Server

The fabric-api server provides REST endpoints for blockchain operations:

5.1.1 Key Endpoints

- **POST /api/landrequests/create**: Create land request
- **POST /api/landrequests/update**: Update request status
- **GET /api/landrequests**: Query land requests
- **GET /api/landrequests/history**: Get action history
- **POST /api/patta/issue**: Issue patta certificate

5.1.2 Role-Based Organization Mapping

```
const orgRoleMap = {
    // Org1
    'clerk': 'org1',
    'superintendent': 'org1',
    'project_officer': 'org1',

    // Org2
    'mro': 'org2',
    'surveyor': 'org2',
    'revenue_inspector': 'org2',
    'vro': 'org2',

    // Org3
    'revenue_dept_officer': 'org3',
    'joint_collector': 'org3',
    'district_collector': 'org3',
    'ministry_welfare': 'org3'
};
```

5.2 Client API Routes

5.2.1 Authentication Routes

- **POST /api/users/register**: User registration
- **POST /api/officials/register**: Official registration
- **POST /api/auth/login**: User login
- **POST /api/officiallogin**: Official login

5.2.2 Land Request Routes

- **POST /api/land-requests/create**: Create land request
- **GET /api/dashboard/applications**: Get assigned applications
- **POST /api/dashboard/action**: Process approval/rejection
- **POST /api/patta/generate**: Generate patta certificate

5.2.3 Document Management

- **POST /api/ipfs/upload**: Upload to IPFS
- **GET /api/documents/view**: View documents
- **POST /api/survey/upload**: Upload survey data

6 Frontend Implementation

6.1 Application Structure

The Next.js application follows the app router architecture:

```
src/app/
          page.tsx                    # Landing page
          layout.tsx                  # Root layout
          globals.css                 # Global styles
          userlogin/                  # User authentication
          userregistration/
          officiallogin/
          officialregistration/
          dashboard/                  # User dashboard
          official-dashboard/         # Official dashboard
          api/                        # API routes
          components/                 # Reusable components
```

6.2 Key Components

6.2.1 Dashboard Components

- **OfficialDashboard**: Main dashboard for officials
- **DashboardHeader**: Navigation header
- **ApplicationHistory**: Action timeline component
- **OTPModal**: OTP verification modal

6.2.2 Role-Specific Components

- **ClerkFields**: Clerk-specific form fields
- **MROFields**: MRO-specific form fields
- **SurveyorFields**: Surveyor-specific form fields
- **DistrictCollectorFields**: District collector fields

6.3 Workflow Implementation

6.3.1 Approval Hierarchy

The system implements a hierarchical approval workflow:

1. **Clerk** → Document intake and initial verification
2. **Superintendent** → Application review
3. **Project Officer** → Project assessment
4. **MRO** → Mandal-level approval
5. **Surveyor** → Land survey and verification
6. **Revenue Inspector** → Revenue record inspection
7. **VRO** → Village-level verification
8. **Revenue Dept Officer** → Revenue department approval
9. **Joint Collector** → District-level oversight
10. **District Collector** → District administration
11. **Ministry of Welfare** → Final approval and patta issuance

6.3.2 Status Management

```
const STATUS_MAP = {
  'clerk': 'with_clerk',
  'superintendent': 'with_superintendent',
  'project_officer': 'with_projectofficer',
  'mro': 'with_mro',
  'surveyor': 'with_surveyor',
  'revenue_inspector': 'with_revenueinspector',
  'vro': 'with_vro',
  'revenue_dept_officer': 'with_revenuedeptofficer',
  'joint_collector': 'with_jointcollector',
  'district_collector': 'with_districtcollector',
  'ministry_welfare': 'with_ministrywelfare'
};
```

7 Deployment and Configuration

7.1 Hyperledger Fabric Network Setup

7.1.1 3-Organization Network Installation

Step 1: Start the Fabric Network with 3 Organizations

```
cd fabric-samples-full/test-network

# Start network with 3 orgs, create channel, use CouchDB
./network.sh up createChannel -ca -s couchdb
```

Step 2: Add Org3 to the Network

```
# Navigate to addOrg3 directory
cd addOrg3

# Add Org3 to existing network
./addOrg3.sh up
```

7.1.2 Chaincode Deployment Process

Step 3: Package the Chaincode

```
# From test-network directory
cd ../test-network

# Package landrecords chaincode
peer lifecycle chaincode package landrecords.tar.gz \
  --path ../../chaincode/landrecords \
  --lang node \
  --label landrecords_1.0
```

Step 4: Install Chaincode on All Organizations
Install on Org1 (Peer0):

```
# Set Org1 environment
export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=$PWD/../config/

# Install on Org1 peer
peer lifecycle chaincode install landrecords.tar.gz
```

Install on Org2 (Peer0):

```
# Switch to Org2
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org2MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org2.example.com/users/Admin@org2.example.com/msp
export CORE_PEER_ADDRESS=localhost:9051

# Install on Org2 peer
peer lifecycle chaincode install landrecords.tar.gz
```

Install on Org3 (Peer0):

```
# Switch to Org3
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org3MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org3.example.com/peers/peer0.org3.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org3.example.com/users/Admin@org3.example.com/msp
export CORE_PEER_ADDRESS=localhost:11051

# Install on Org3 peer
peer lifecycle chaincode install landrecords.tar.gz
```

Step 5: Approve Chaincode for All Organizations
Approve for Org1:

```
# Switch back to Org1
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

# Approve for Org1
peer lifecycle chaincode approveformyorg \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --channelID mychannel \
  --name landrecords \
  --version 1.0 \
  --package-id $CC_PACKAGE_ID \
  --sequence 1 \
  --tls \
  --cafile ${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

Approve for Org2:

```
# Switch to Org2
export CORE_PEER_TLS_ENABLED=true
```

```
export CORE_PEER_LOCALMSPID="Org2MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/
    peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/
    ca.crt
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/
    org2.example.com/users/Admin@org2.example.com/msp
export CORE_PEER_ADDRESS=localhost:9051

# Approve for Org2
peer lifecycle chaincode approveformyorg \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --channelID mychannel \
  --name landrecords \
  --version 1.0 \
  --package-id $CC_PACKAGE_ID \
  --sequence 1 \
  --tls \
  --cafile ${PWD}/organizations/ordererOrganizations/example.com/
    orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.
    pem
```

Approve for Org3:

```
# Switch to Org3
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org3MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/
    peerOrganizations/org3.example.com/peers/peer0.org3.example.com/tls/
    ca.crt
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/
    org3.example.com/users/Admin@org3.example.com/msp
export CORE_PEER_ADDRESS=localhost:11051

# Approve for Org3
peer lifecycle chaincode approveformyorg \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --channelID mychannel \
  --name landrecords \
  --version 1.0 \
  --package-id $CC_PACKAGE_ID \
  --sequence 1 \
  --tls \
  --cafile ${PWD}/organizations/ordererOrganizations/example.com/
    orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.
    pem
```

Step 6: Commit Chaincode to Channel

```
# Switch back to Org1 for commit
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/
    peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/
    ca.crt
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/
    org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

```

```
# Commit chaincode
peer lifecycle chaincode commit \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --channelID mychannel \
  --name landrecords \
  --version 1.0 \
  --sequence 1 \
  --tls \
  --cafile ${PWD}/organizations/ordererOrganizations/example.com/
   orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.
   pem \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles ${PWD}/organizations/peerOrganizations/org1.
   example.com/peers/peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles ${PWD}/organizations/peerOrganizations/org2.
   example.com/peers/peer0.org2.example.com/tls/ca.crt \
  --peerAddresses localhost:11051 \
  --tlsRootCertFiles ${PWD}/organizations/peerOrganizations/org3.
   example.com/peers/peer0.org3.example.com/tls/ca.crt
```

7.1.3 Verification Commands

Check Chaincode Status:

```
peer lifecycle chaincode querycommitted \
  --channelID mychannel \
  --name landrecords
```

Test Chaincode:

```
# Query all land requests (should return empty initially)
peer chaincode query \
  -C mychannel \
  -n landrecords \
  -c '{"Args":["getAllLandRequests"]}'
```

7.2 Application Deployment

7.2.1 Environment Configuration

```
# Client (.env.local)
MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/eland-
   records
NEXT_PUBLIC_API_URL=http://localhost:3000
PINATA_JWT=your_pinata_jwt_token

# Fabric-API (.env)
MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/eland-
   records
```

7.2.2 Service Startup

```
# Terminal 1: Fabric-API Server
cd fabric-api
node api.js

# Terminal 2: Client Application
cd client
npm run dev

# Terminal 3: MongoDB Server (if using local)
mongod
```

7.2.3 Access URLs

- **Web Application**: http://localhost:3000
- **Fabric-API**: http://localhost:4040
- **MongoDB**: mongodb://localhost:27017

8 Security Implementation

8.1 Authentication and Authorization

8.1.1 Password Security

- Bcrypt hashing with salt rounds
- Secure password policies
- Session-based authentication

8.1.2 Role-Based Access Control

- 11 distinct official roles
- Organization-based permissions
- API-level authorization checks

8.1.3 Blockchain Security

- Cryptographic signatures for all transactions
- Immutable audit trails
- Decentralized consensus
- Private data collections for sensitive information

8.2 Data Protection

8.2.1 Document Security

- IPFS decentralized storage
- SHA-256 hash verification
- Access control based on user roles

8.2.2 Network Security

- TLS encryption for all communications
- Certificate-based authentication
- Private channels for sensitive operations

9 Testing and Validation

9.1 Unit Testing

- Chaincode function testing
- API endpoint validation
- Component testing with Jest
- Database operation verification

9.2 Integration Testing

- End-to-end workflow testing
- Cross-service communication
- Blockchain synchronization validation
- Document upload and retrieval

9.3 Performance Testing

- Concurrent user load testing
- Blockchain transaction throughput
- Database query performance
- File upload/download speeds

10 Challenges and Solutions

10.1 Technical Challenges

10.1.1 Chaincode Deployment Issues

Problem: Metadata file path validation errors during packaging **Solution**: Corrected META-INF directory structure and added CouchDB indexes

10.1.2 Organization Migration

Problem: Org3 setup failures requiring user migration **Solution**: Implemented flexible role mapping and user reassignment

10.1.3 Blockchain Synchronization

Problem: Ensuring MongoDB and Fabric data consistency **Solution**: Implemented dual-write pattern with error handling

10.1.4 Cross-Service Communication

Problem: Coordinating between Next.js, Fabric-API, and blockchain **Solution**: RESTful API design with comprehensive error handling

10.2 Business Challenges

10.2.1 Complex Approval Hierarchy

Problem: Managing 11-tier approval workflow **Solution**: Automated status transitions and role-based routing

10.2.2 Document Management

Problem: Secure storage and verification of land documents **Solution**: IPFS integration with blockchain hash verification

10.2.3 User Experience

Problem: Complex forms and workflows for end users **Solution**: Progressive disclosure and contextual help

11 Future Enhancements

11.1 Advanced Features

- **Mobile Application**: Native mobile apps for field officials
- **AI-Powered Verification**: Automated document verification
- **Integration APIs**: Third-party system integrations
- **Advanced Analytics**: Dashboard with insights and reports
- **Multi-language Support**: Localization for regional languages

11.2 Technical Improvements

- **Microservices Architecture**: Decompose monolithic services
- **Event-Driven Architecture**: Real-time notifications
- **Advanced Security**: Zero-knowledge proofs and encryption
- **Scalability**: Horizontal scaling and load balancing
- **Monitoring**: Comprehensive logging and metrics

11.3 Regulatory Compliance

- **GDPR Compliance**: Data protection and privacy
- **Audit Trails**: Enhanced compliance reporting
- **Digital Signatures**: Legally binding electronic signatures
- **Regulatory Reporting**: Automated compliance reports

12 Conclusion

The E-Land Records Management System successfully demonstrates the practical application of blockchain technology in government land administration. The implementation provides:

- **Secure and Transparent**: Blockchain ensures data integrity and auditability
- **Efficient Workflow**: Automated approval processes reduce processing time
- **User-Friendly Interface**: Intuitive design for both citizens and officials
- **Scalable Architecture**: Modular design supports future enhancements
- **Robust Security**: Multi-layer security with role-based access control

The system addresses critical pain points in traditional land management systems while establishing a foundation for digital transformation in government services. The successful integration of Hyperledger Fabric with modern web technologies proves the viability of blockchain solutions for enterprise applications.

12.1 Lessons Learned

1. Blockchain integration requires careful planning for data synchronization
2. Role-based access control is crucial for multi-organization systems
3. User experience design is essential for complex administrative workflows
4. Comprehensive testing is vital for distributed systems
5. Documentation and monitoring are critical for system maintenance

12.2 Impact and Benefits

The implementation provides significant benefits:

- **Transparency**: Citizens can verify land records independently
- **Efficiency**: Reduced processing time from weeks to days
- **Cost Savings**: Digital processes eliminate paper-based workflows
- **Trust**: Immutable records prevent disputes and corruption
- **Accessibility**: Online services available 24/7

This project establishes a blueprint for blockchain-based government applications and demonstrates the potential for technology to transform public services.

13 Appendices

13.1 Appendix A: Installation and Setup Guide

13.1.1 Prerequisites

1. Node.js 18+ installed
2. Docker and Docker Compose installed
3. Git installed
4. MongoDB Atlas account or local MongoDB
5. IPFS/Pinata account for document storage

13.1.2 Complete Installation Steps

1. Clone and Setup Project

```
git clone <repository-url>
cd e-land-records
```

2. Install Dependencies

```
# Client dependencies
cd client
npm install

# Fabric-API dependencies
cd ../fabric-api
npm install
```

3. Environment Configuration

```
# Client .env.local
MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/eland-
    records
NEXT_PUBLIC_API_URL=http://localhost:3000
PINATA_JWT=your_pinata_jwt_token

# Fabric-API .env
MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/eland-
    records
```

4. Deploy Hyperledger Fabric Network Follow the detailed 3-organization network setup and chaincode deployment steps in Section 6.1.

5. Start All Services

```
# Terminal 1: Fabric-API
cd fabric-api
node api.js

# Terminal 2: Client Application
cd client
npm run dev

# Terminal 3: MongoDB (if using local)
mongod
```

13.1.3 Troubleshooting

Common Issues:

- **Port conflicts**: Change ports in configuration files
- **MongoDB connection**: Verify connection string and network access
- **Fabric network**: Ensure Docker is running and ports are available
- **Chaincode deployment**: Check logs for packaging errors
- **Org3 setup**: Ensure addOrg3.sh completes successfully

13.2 Appendix B: API Endpoints Summary

13.2.1 Fabric-API Endpoints

- **POST /api/landrequests/create**: Create land request on blockchain
- **POST /api/landrequests/update**: Update request status
- **GET /api/landrequests**: Query land requests by role
- **POST /api/patta/issue**: Issue patta certificate

13.2.2 Client API Endpoints

- **POST /api/users/register**: User registration
- **POST /api/officials/register**: Official registration
- **POST /api/land-requests/create**: Create land application
- **POST /api/dashboard/action**: Process approval/rejection
- **POST /api/patta/generate**: Generate patta certificate